

CS 201: Data Structures

Homework 3

Name:

Student ID:

Linked Lists, Trees, and Algorithm Analysis

Due Date October 15, 2026

Instructor Prof. Jane Smith

Total Points 60

Exercise 1 (5 points)

Define a *singly linked list* and state two advantages it has over a dynamic array.

Solution:

A singly linked list is a linear data structure in which each element (node) contains a data field and a reference (pointer) to the next node in the sequence.

Two advantages over a dynamic array:

1. $O(1)$ insertion and deletion at the head (no shifting of elements).
2. No need to pre-allocate or resize a contiguous block of memory.

Exercise 2 (10 points)

Compute the time complexity of the following function using Big-O notation.

```
function mystery(n):  
    x = 0  
    for i = 1 to n:  
        for j = 1 to i:  
            x = x + 1  
    return x
```

Solution:

The inner loop runs i times for each value of i from 1 to n . The total number of iterations is:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore the time complexity is $O(n^2)$.

Exercise 3 (15 points)

Prove by induction that a binary tree of height h has at most $2^{h+1} - 1$ nodes.

Solution:

Base case ($h = 0$): A tree of height 0 is a single node. $2^{0+1} - 1 = 1$. $\square$

Inductive step: Assume a binary tree of height h has at most $2^{h+1} - 1$ nodes. Consider a tree of height $h+1$. Its left and right subtrees each have height at most h , so by the inductive hypothesis each

has at most $2^{h+1} - 1$ nodes. Adding the root:

$$1 + 2(2^{h+1} - 1) = 1 + 2^{h+2} - 2 = 2^{h+2} - 1$$

This completes the induction.

Exercise 4 (10 points)

Write pseudocode for an in-order traversal of a binary search tree that collects all values into an array.

Solution:

```
function inorder(root, result):
    if root == null:
        return
    inorder(root.left, result)
    append(result, root.value)
    inorder(root.right, result)
```

For a BST with n nodes this runs in $O(n)$ time and uses $O(h)$ stack space, where h is the tree height.

Exercise 5 (15 points)

Consider a hash table that uses separate chaining with m slots and n keys. Answer the following:

- (a) What is the expected length of a chain under simple uniform hashing?
- (b) If $n = \alpha m$ for load factor α , what is the expected time for an unsuccessful search?
- (c) How does open addressing (linear probing) differ in its expected search time?

Solution:

- (a) Under simple uniform hashing the expected chain length is $\frac{n}{m} = \alpha$, the load factor.
- (b) An unsuccessful search must examine the entire chain on average, giving $O(1 + \alpha)$ expected time.
- (c) With linear probing the expected number of probes for an unsuccessful search is approximately $\frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$ for $\alpha < 1$, which grows faster than separate chaining as $\alpha \rightarrow 1$.

Exercise 6 (5 points)

State the Master Theorem. You do not need to prove it.

Solution:

For a recurrence of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$, and f is asymptotically positive:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the regularity condition holds, then $T(n) = \Theta(f(n))$.